Fakultas Informatika
School of Computing
Telkom University

Pemrograman Berorientasi Objek

Class Diagram

Overview

- UML
- Modelling Class in Class Diagram
- Modelling Class Relationship in Class Diagram

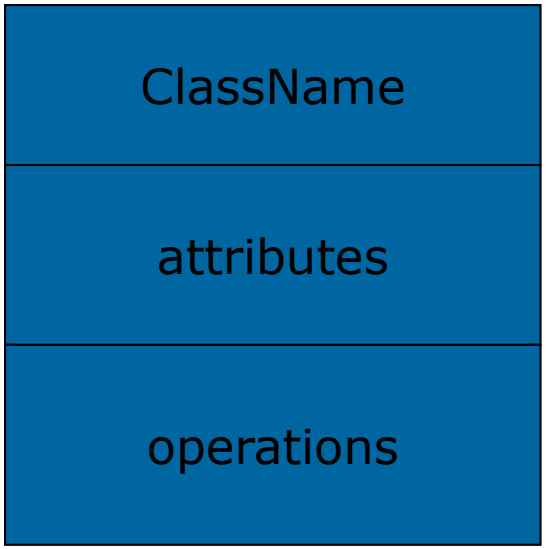
Unified Modeling Language

- Unified Modeling Language (UML) is a standardized general-purpose modeling language in the field of software engineering. The standard is managed, and was created by, the Object Management Group.
- UML includes a set of graphic notation techniques to create visual models of software-intensive systems.
- To model the classes and their relationship, in UML we use class diagram

Why we use UML?

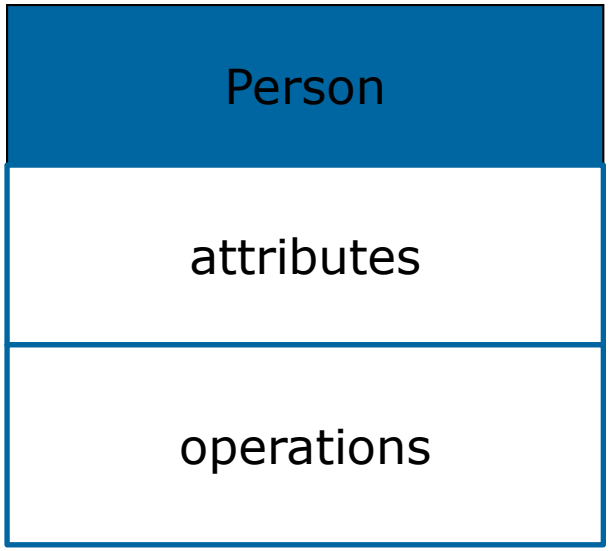
- Language: express idea, not a methodology
- Modeling: Describing a software system at a high level of abstraction
- Unified: a world standard
  - Object Management Group (OMG): www.omg.org

Modelling Class in Class Diagram



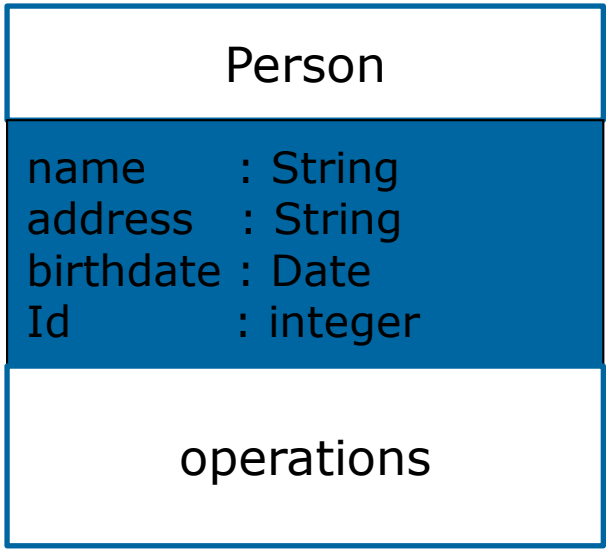
- A class is a description of a set of objects that share the same attributes, operations, relationships, and semantics.

ClassName



- The name of the class is the only required tag in the graphical representation of a class.
- It always appears in the top-most compartment.

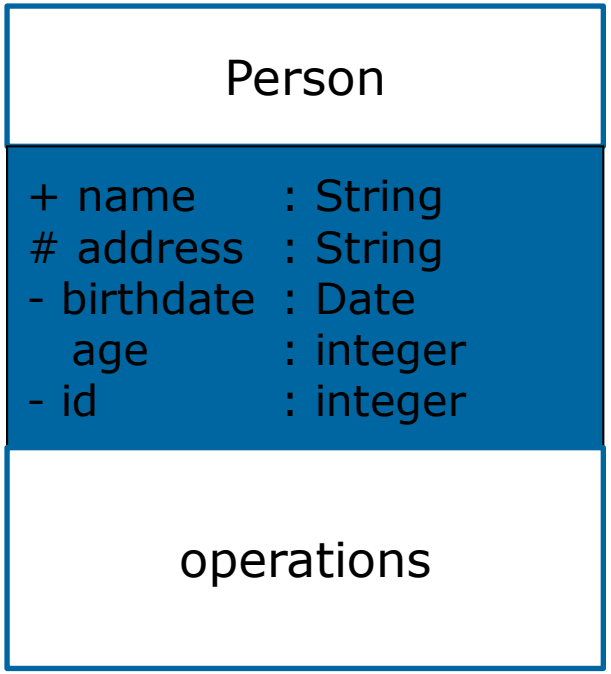
Class Attributes



- An attribute is a named property of a class that describes the object being modeled.
- In the class diagram, attributes appear in the second compartment just below the name-compartment.
- Attributes are usually listed in the form:

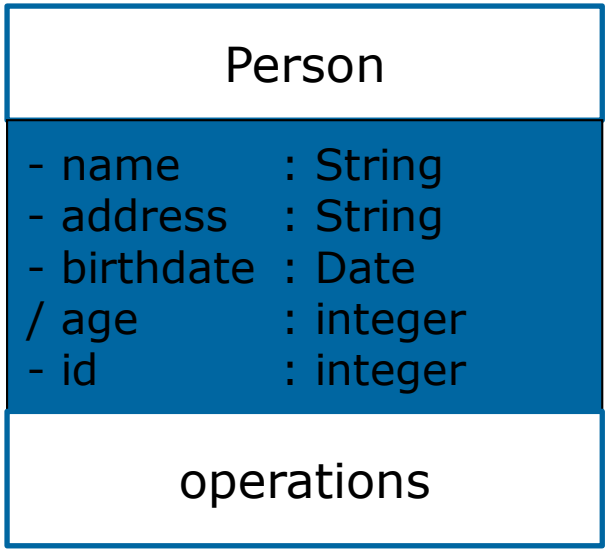
  attributeName : Type

Class Attributes



- Attributes can be:
  - + (public)
  - # (protected)
  - - (private)
- Normally use private for field

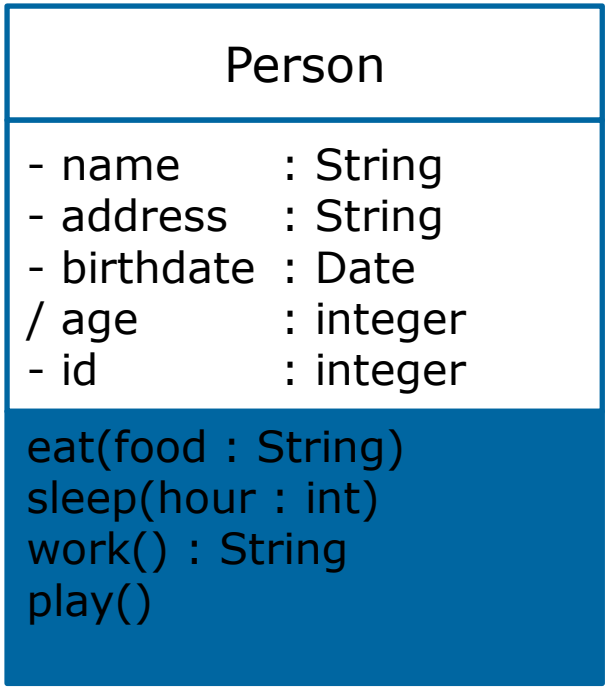
Class Attributes



- A derived attribute is one that can be computed from other attributes, but doesn't actually exist. For example, a Person's age can be computed from his birth date.
- A derived attribute is designated by a preceding '/' as in:

  / age : Date

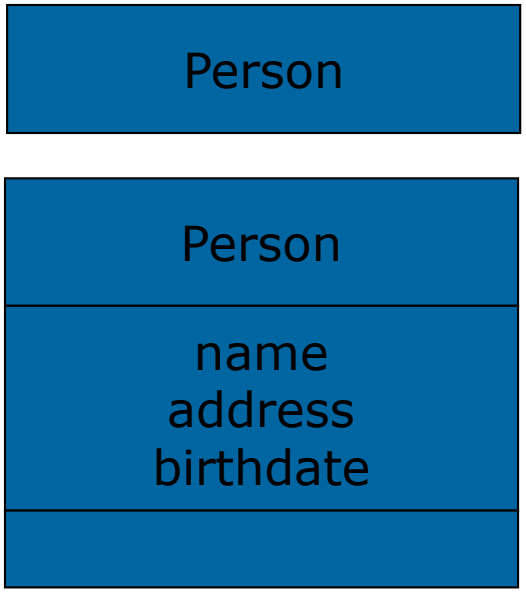
Class Operations

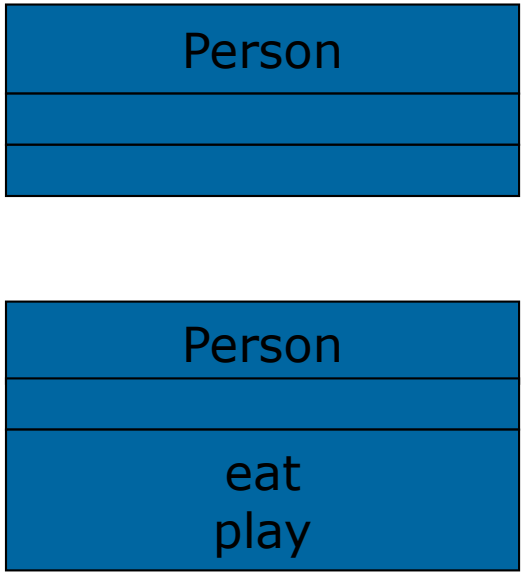


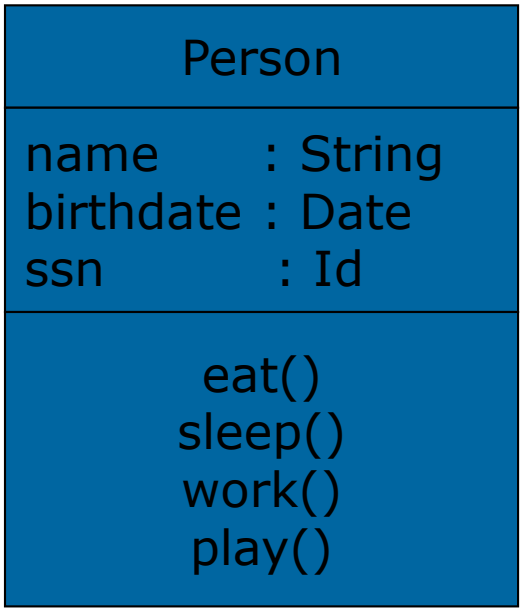
- Operations describe the class behavior and appear in the third compartment
- You can specify an operation by stating its signature:
  - listing the name, type, and default value of all parameters, and, in the case of functions, a return type.

Depicting Classes

- When drawing a class, you needn't show attributes and operation in every diagram





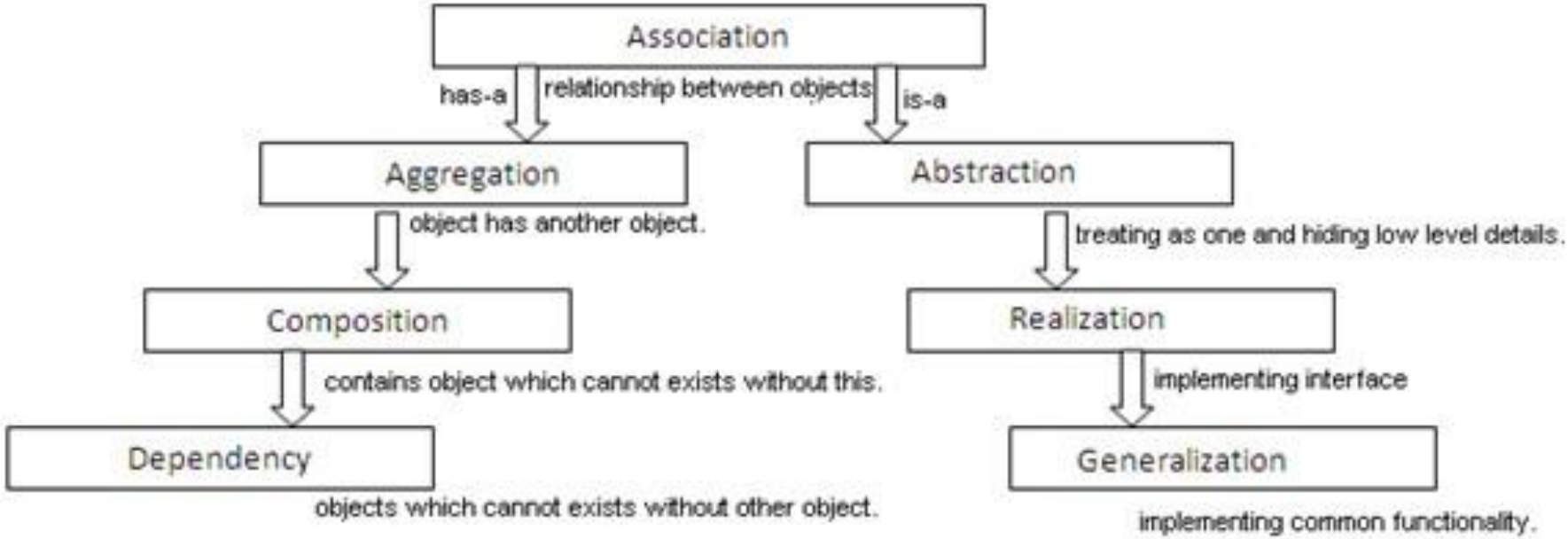


Question?

Fakultas Informatika
School of Computing
Telkom University

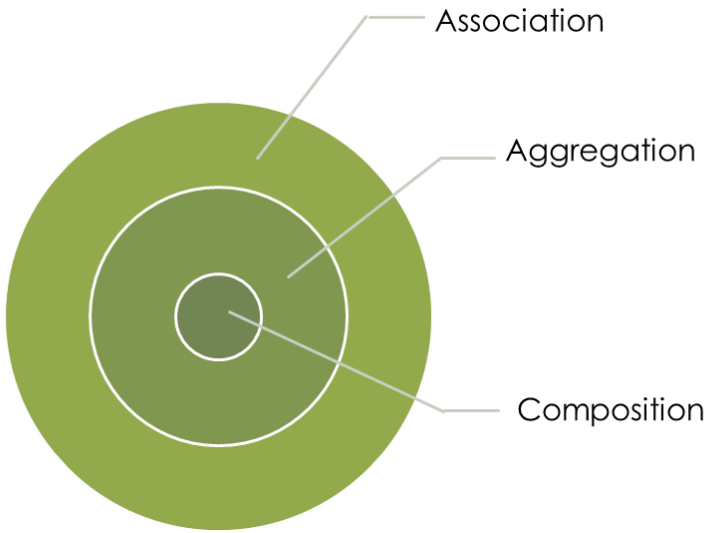
Modelling Class Relationship in Class Diagram

Class Association



Basic Relationship

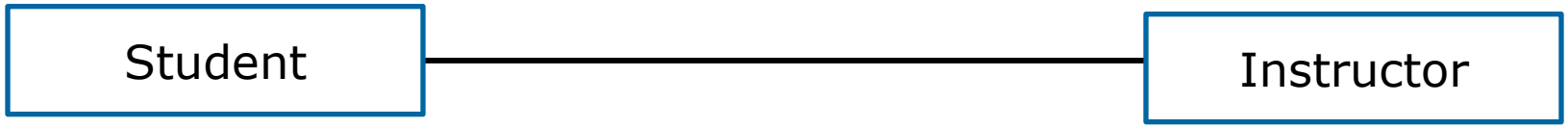
- Association
- Aggregation
- Composition



Association

Association Relationships

- If two classes in a model need to **communicate** with each other, there must be link between them.
- An association denotes that link.



- An association between two classes indicates that objects at one end of an association **"recognize"** objects at the other end and may send messages to them.

Example

```
public class Student{
   private String name;
   private String assignment;

   public Student(String name){
      this.name = name;
   }
   public void setName(String name){
      this.name = name;
   }
   public String getName(){
      return name;
   }
   public void setAssignment(String assignment){
      this.assignment = assignment;
   }

   public String getAssignment(){
      return assignment;
   }
}
```

Example

```
public class Instructor{
   private String name;

   public Instructor(String name){
       this.name = name;
   }
   public void giveAssignment(Student s, String assignment){
       s.setAssignment(assignment);
   }
}
```

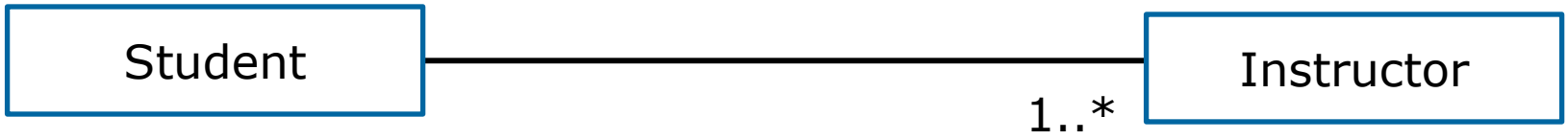
```
public class Driver{
   public static void main(String args[]){
       Student s1 = new Student("andi");
       Instructor i1 = new Instructor("budi");
       i1.giveAssignment(s1,"code java OOP");
       System.out.println(s1.getAssignment());
   }
}
```

Association Relationships

- Message passing
  - Object sends data to another object
- Method calling
  - Use or ask the object to invoke a method

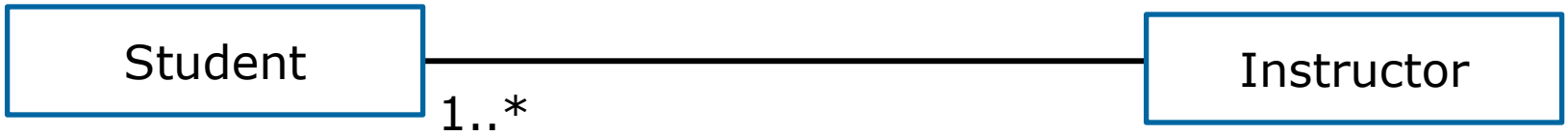
Association Relationships

- We can indicate the multiplicity of an association by adding multiplicity adornments to the line denoting the association.
- The example indicates that a Student has one or more Instructors:



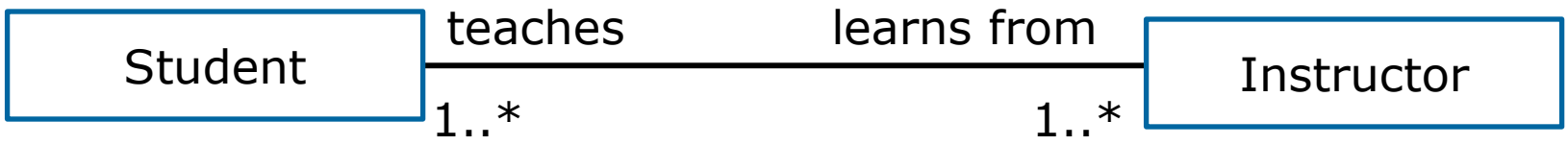
Association Relationships

- The example indicates that every Instructor has one or more Students:



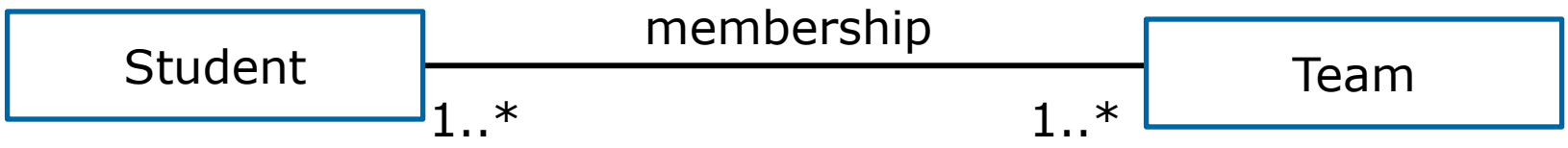
Association Relationships

- We can also indicate the behavior of an object in an association (i.e., the role of an object) using role names.



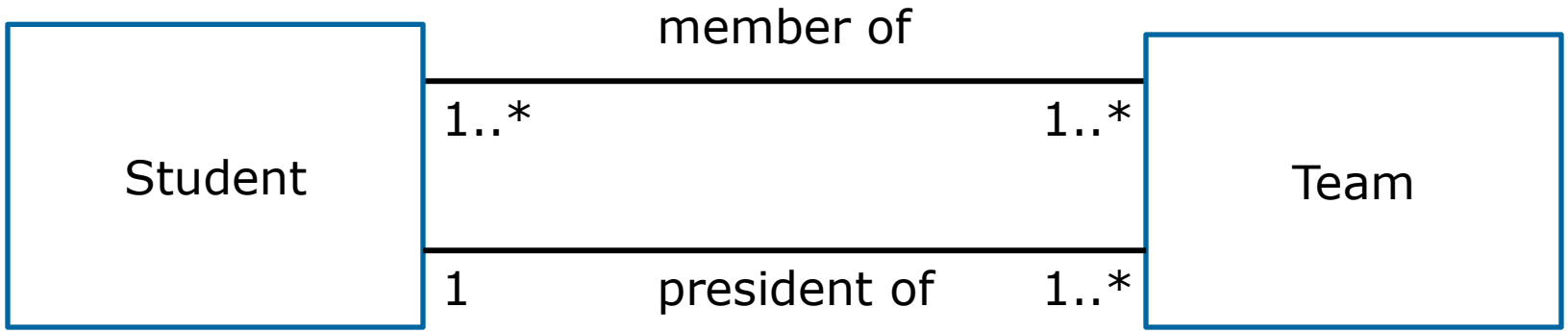
Association Relationships

- We can also name the association.



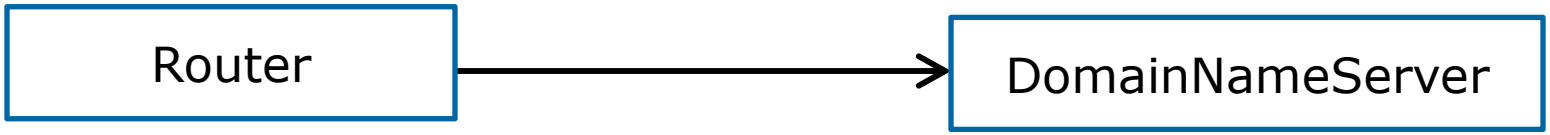
Association Relationships

- We can specify dual associations.



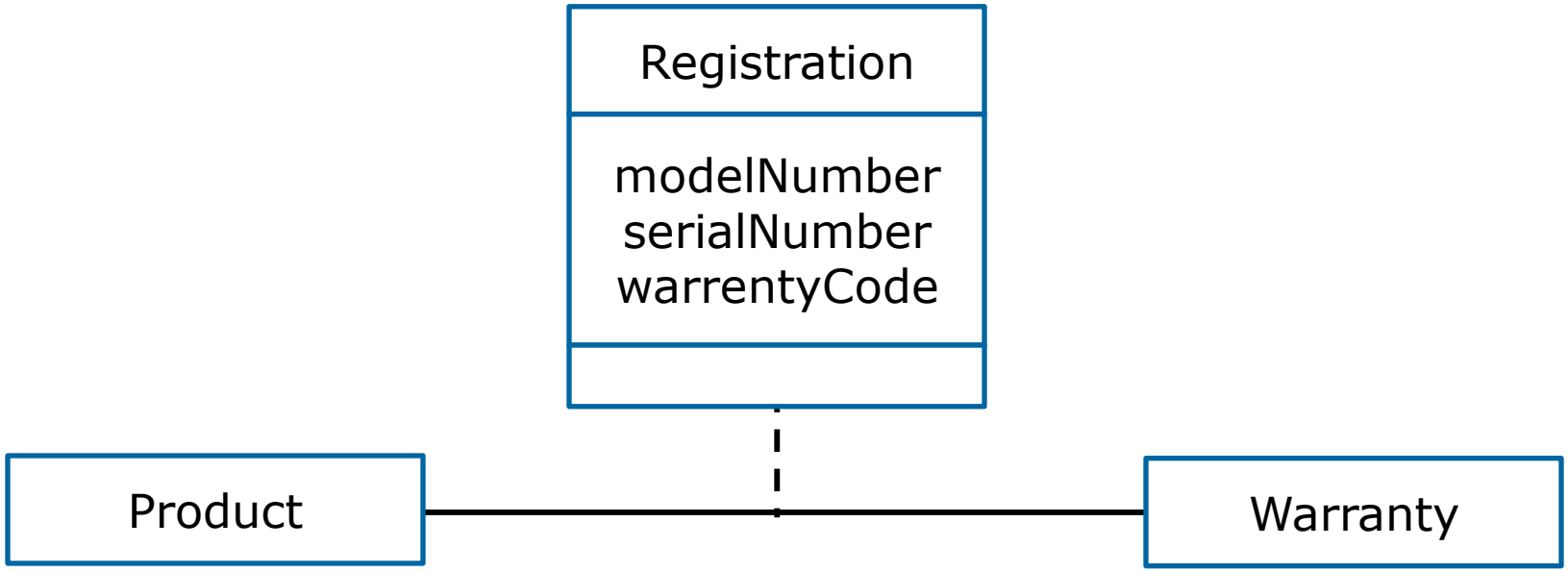
Association Relationships

- We can constrain the association relationship by defining the navigability of the association.
  - Here, a Router object requests services from a DNS object by sending messages to (invoking the operations of) the server.
  - The direction of the association indicates that the server has no knowledge of the Router.



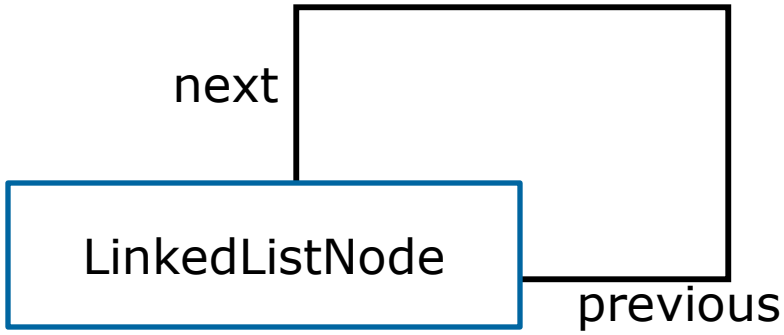
Association Relationships

- Associations can also be objects themselves, called link classes or an association classes.



Association Relationships

- A class can have a self association.



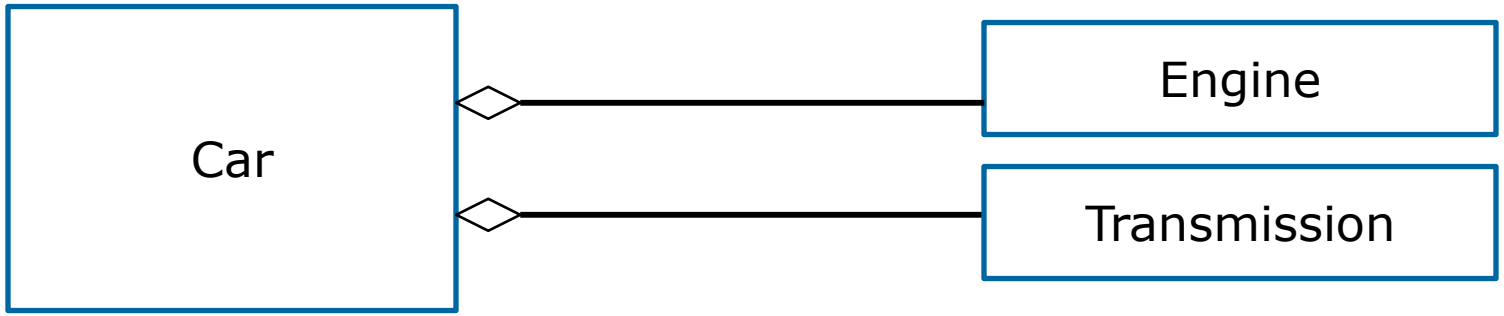
Aggregation

Aggregation Relationships

- We can model objects that contain other objects by way of special associations called aggregations and compositions.
- An aggregation specifies **a whole-part** relationship between an aggregate (a whole) and a constituent part, where the part can exist independently from the aggregate.

Aggregation Relationships

- Aggregation is a variant of the **"has a"** or association relationship.
- Aggregations are denoted by a hollow-diamond adornment on the association.



Example

```
public class Engine {
    private String name;
    private int horsePower;

    public Engine(String name){
        this.name = name;
    }

    public int getHorsePower() {
        return horsePower;
    }

    public void setHorsePower(int hp) {
        this.horsePower = hp;
    }
}
```

```
public class Transmission {
    private String type;

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }
}
```

Example

```
public class Car {
    private String name;
    private Engine engine;
    private Transmission transmission;

    public Car(String name){
        this.name = name;
    }

    public void addEngine(Engine e){
        engine = e;
    }

    public void addTransmission(Transmission t){
        transmission = t;
    }
}
```

Example

```
public class Driver {

    public static void main(String[] args) {
        Car c = new Car("honda");

        Engine v1000 = new Engine("v1000");

        Transmisson auto = new Transmisson();
        auto.setType("Automatic");

        c.addEngine(v1000);
        c.addTransmission(auto);

    }
}
```

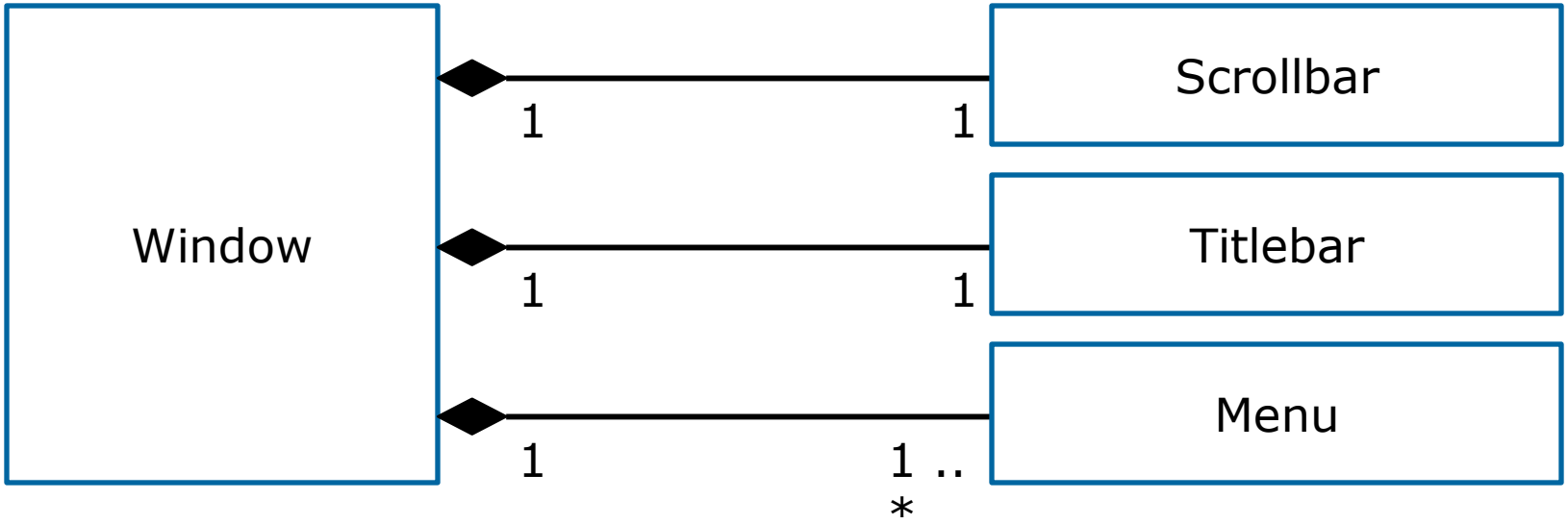
Composition

Composition Relationships

- A stronger variant of the has-a relationship, which is the **"owns a"** relationship
- A composition indicates a strong ownership and coincident lifetime of parts by the whole (i.e., they live and die as a whole).

Composition Relationships

- Compositions are denoted by a filled-diamond adornment on the association.



Example

```
public class Scrollbar {
    public String type;

    public Scrollbar(String type){
        this.type = type;
    }
}
```

```
public class Titlebar {
    public String title;

    public Titlebar(String title){
        this.title = title;
    }
}
```

```
public class Menu {
    private String title;
    private String type;

    public Menu(String title, String type){
        this.title = title;
        this.type = type;
    }
}
```

Example

```
public class Window {
    private Scrollbar scBar;
    private Titlebar tlBar;
    private Menu[] menu;

    public Window(String title, String scrollType, int numMenu){
        scBar = new Scrollbar(scrollType);
        tlBar = new Titlebar(title);
        menu = new Menu[numMenu];
    }
}
```

```
public class Driver {
    public static void main(String[] args) {
        Window w = new Window("OOP Window", "vertical", 5);
    }
}
```

Example

Specify the appropriate relation to each case:

1. A person and a car that he wants to buy
2. A car in a parking lot
3. A mall and its basement
4. Wheels in a car
5. A department and a company
6. A department and an employee
7. A canteen and a department
8. A pond and fishes

Question?

Fakultas Informatika
School of Computing
Telkom University

THANK YOU